Project Briefing & Dissertation Blueprint: Reinforcement Learning-Based Adaptive Modulation and Coding (v6)

RL-AMC Case Study – Technical Report & Thesis Briefing Document

July 15, 2026

Abstract

This briefing document compiles the complete evolution, design audit, mathematical formulations, and multi-sweep empirical results of the v6 OLLA-relative Reinforcement Learning-based Adaptive Modulation and Coding (RL-AMC) project. It serves as a master blueprint for structuring a Master's or Ph.D. dissertation. The project models link adaptation in a multi-user, mobile LTE/5G-NR downlink as a Partially Observable Markov Decision Process (POMDP). We detail the design audit that drove the development from v1 to v6, resolving critical simulator bugs (scheduler information leaks, reward prisoner's dilemmas). Empirical validation across 5 seeds and 30 traces demonstrates that Deep A2C statistically outperforms OLLA at delays of 2 and 6 TTIs. Under 30 m/s high-mobility velocity sweeps, the coherence time drops below the feedback lag, expanding the Deep A2C margin to +9.4%. In addition to presenting these results, this report embeds 15 primary evaluation plots (delay sweeps, learning curves, robustness ratios, BLER, fairness, edge throughput, user-level distance regressions, trace-level win-rates, action distributions, user scaling, and action offset expansion sweeps) and outlines a structured chapter-by-chapter thesis narrative.

1 Introduction, Goal, and Research Process

The core goal of this research is to design and evaluate **Reinforcement Learning (RL) algorithms for Link Adaptation (LA)** in a multi-user, mobile cellular downlink. Specifically, the task is **Adaptive Modulation and Coding (AMC)**: dynamically selecting the Modulation and Coding Scheme (MCS) for each scheduled user at each Transmission Time Interval (TTI).

The research process followed a rigorous **audit-driven evolution**:

1. We implemented standard textbook RL-AMC models, which appeared to outperform baseline models.
2. We executed a comprehensive design audit that identified simulator design flaws that leaked information or decoupled rates and error rewards.
3. We resolved these flaws, which collapsed standard RL performance, forcing us to rebuild the models.
4. We addressed the remaining informational asymmetry by structuring the RL agents to act relative to OLLA (v6), leading to genuine, statistically significant gains.

2 The POMDP Mathematical Formulation

Because the Proportional Fair (PF) scheduling potential depends on the rates and channel states of all users, a single-user MDP is insufficient. The multi-user cell is modeled as a Partially Observable Markov Decision Process (POMDP).

2.1 State Vector and Normalization

The observed state vector $\mathbf{s}_t(u)$ for user u at interval t is:

$$\mathbf{s}_t(u) = \left[\frac{\text{ref_CQI}_t(u)}{15}, \frac{d_ref_t(u)}{5}, \frac{\text{obs_SINR}_t(u)}{30}, \frac{\text{obs_SINR_trend}_t(u)}{5}, \frac{\Delta_t(u)}{5}, \mathbf{c}_{\text{sched}} \right]^T \tag{1}$$

where $\text{ref_CQI}_t(u)$ is the OLLA-corrected reference CQI, $d_ref_t(u)$ is the tracking velocity, $\text{obs_SINR}_t(u)$ is the noisy reported SINR, $\text{obs_SINR_trend}_t(u)$ is the observed trend, $\Delta_t(u)$ is the OLLA loop offset, and $\mathbf{c}_{\text{sched}}$ is a one-hot scheduling context.

2.2 Cooperative Centralized Reward

To avoid competitive over-reporting where users artificially inflate reports to win scheduling slots, the cell goodput is aggregated cooperatively:

$$r_t = \frac{\sum_{u \in \mathcal{S}_t} (1 - \text{BLER}_t(u)) \times \text{TBS}_t(u)}{\rho} \tag{2}$$

where $\mathcal{S}_t$ is the scheduled user set, and $\rho = 30$ Mbps normalizes the reward.

3 Outer Loop Link Adaptation (OLLA) and its Selection as the Primary Benchmark

3.1 What is OLLA?

Outer Loop Link Adaptation (OLLA) is a closed-loop control system widely deployed in 4G LTE and 5G NR base stations to dynamically adjust the Modulation and Coding Scheme (MCS) selection. The base station first maps the user's reported Channel Quality Indicator (CQI) to an expected signal-to-interference-plus-noise ratio (SINR). Because reported CQIs are corrupted by feedback delay, measurement noise, and fast fading, OLLA introduces a dynamic offset Δ_{OLLA} that is added to the reported SINR before MCS selection:

$$\text{effective_SINR}_t = \text{reported_SINR}_t + \Delta_{\text{OLLA}} \tag{3}$$

The offset Δ_{OLLA} is updated dynamically based on Hybrid Automatic Repeat Request (HARQ) acknowledgment (ACK/NACK) feedback from the receiver:

- **On Success (ACK):** The offset is incremented slightly to encourage a higher MCS rate:

$$\Delta_{\text{OLLA}} \leftarrow \Delta_{\text{OLLA}} + \delta_{\text{up}} \tag{4}$$

- **On Failure (NACK):** The offset is decremented to back off and reduce transmission errors:

$$\Delta_{\text{OLLA}} \leftarrow \Delta_{\text{OLLA}} - \delta_{\text{down}} \tag{5}$$

To guarantee that the block error rate (BLER) converges to a target rate P_{target} (typically 10%), the steps satisfy the balance equation:

$$(1 - P_{\text{target}}) \times \delta_{\text{up}} = P_{\text{target}} \times \delta_{\text{down}} \implies \delta_{\text{up}} = \delta_{\text{down}} \frac{P_{\text{target}}}{1 - P_{\text{target}}} \tag{6}$$

3.2 Why OLLA is the Benchmark Instead of the True Baseline in v6

In initial project iterations (v1–v4), the "true baseline" (a static, open-loop lookup table mapping SINR directly to CQI without dynamic feedback) was used as the primary comparator. While reinforcement learning models easily outperformed this static baseline, it represents a weaker benchmark. Modern cellular networks never utilize open-loop mapping due to its inability to handle fast fading, modeling errors, or interference changes. Comparing against a static baseline overstates the utility of RL.

OLLA represents the industry standard. However, comparing RL to OLLA in an absolute action space (as in v5) results in a stalemate where RL matches but cannot beat OLLA. This is due to *informational asymmetry*: OLLA has direct access to ACK/NACK feedback, which is not represented in the RL agent's state vector.

In v6, we resolved this by shifting from an absolute action space to an **OLLA-relative action space**. The reference CQI is OLLA-corrected, and the RL agent outputs an offset relative to OLLA. In this configuration, OLLA acts as a pre-conditioned, stable base controller that manages steady-state error correction. The RL agent's task is simplified to learning *proactive corrections* (such as predicting Doppler-induced fading trends) that OLLA cannot resolve due to its reactive nature. Choosing OLLA as the comparative benchmark is therefore the only scientifically rigorous way to prove that the RL agent provides performance gains over state-of-the-art industrial standards.

3.3 Operational Intuition

OLLA acts as a reactive closed-loop controller that adjusts transmission parameters based on past success or failure, similar to a thermostat regulating temperature. The open-loop true baseline, by contrast, operates on static mappings without feedback, rendering it an inadequate representative of modern industrial practices. By defining the reinforcement learning policy relative to the OLLA-corrected state, OLLA handles steady-state regulation while the learning agent specializes in predictive corrections, ensuring a rigorous, state-of-the-art comparison.

4 Detailed Version Evolution & Audit History

4.1 v1: Per-User Absolute Actions

- **Defect:** Tabular and deep models collapsed far below baseline (e.g., Q-Learning was 3.93 Mbps vs. Baseline's 9.11 Mbps).
- **Audit Cause:** (a) Zero-init argmax tie-break bias in MATLAB's `max` function drove actions to the minimum offset. (b) Per-user tables fragmented experience. (c) Case 1 reward was decoupled (TBS read from baseline while BLER charged to agent).
- **Fix:** Aligned TBS and BLER to agent actions, and implemented `argmaxNoOp` tie-breaking.

4.2 v2–v3: No-Op Bias and Shaping

- **Defect:** Strongly biased models performed worst (Tabular Actor-Critic collapsed to 2.55 Mbps).
- **Audit Cause:** The *no-op imitation fallacy*. Since actions are offsets, offset 0 freezes the CQI instead of tracking SINR, preventing channel tracking.
- **Fix:** Deleted the no-op bias and shaping bonuses.

4.3 v4: Baseline-Relative Rewrite

- **Concept:** Action is defined as an offset from a fixed-mapping reference (offset $0 \equiv$ baseline).
- **Outcome:** Deep A2C beat OLLA (12.32 vs. 12.08 Mbps).
- **Audit Fixes:** Calibrated BLER sigmoid waterfalls, introduced noise/delay (imperfect CQI), and normalized training parameters (bootstrapping, advantage standardization).

4.4 v5: Honest Scheduler and Cooperative Reward

- **Defect:** Re-auditing v4 revealed that OLLA was reaching +164% over the fixed baseline—a physical impossibility.
- **Audit Cause:** (a) Oracle Scheduler Leak: the scheduler potential was computed from future true BLER, shielding aggressive policies. (b) Selfish Reward Prisoner's Dilemma: per-user rewards incentivized CQI over-reporting.
- **Fix:** Modified scheduler to read reported CQI only (`pf_metric = 'reported'`) and introduced cooperative cell reward (`reward_agg = 'cell'`).

4.5 v6: OLLA-Relative Action Space

- **Defect:** In v5, RL matched but did not beat OLLA because OLLA utilized ACK/NACK feedback that RL could not access.
- **Fix:** Integrated OLLA loop inside the learners. Reference CQI = `baseCQI(obs_SINR + olla_offset)`; action is an offset from OLLA. Exposes the OLLA offset as a state feature.

---

5 Experimental Results & Multi-Sweep Analysis

5.1 Feedback Delay Sweep

Table 1 shows the PF cell throughput points. The Deep A2C margin over OLLA peaks at **delay 6** (+6.7%), surviving Holm-Bonferroni correction. Figure 1 visualizes this delay sweep.

Fig. 4 Throughput vs CQI feedback delay (PF, v6, N=10)

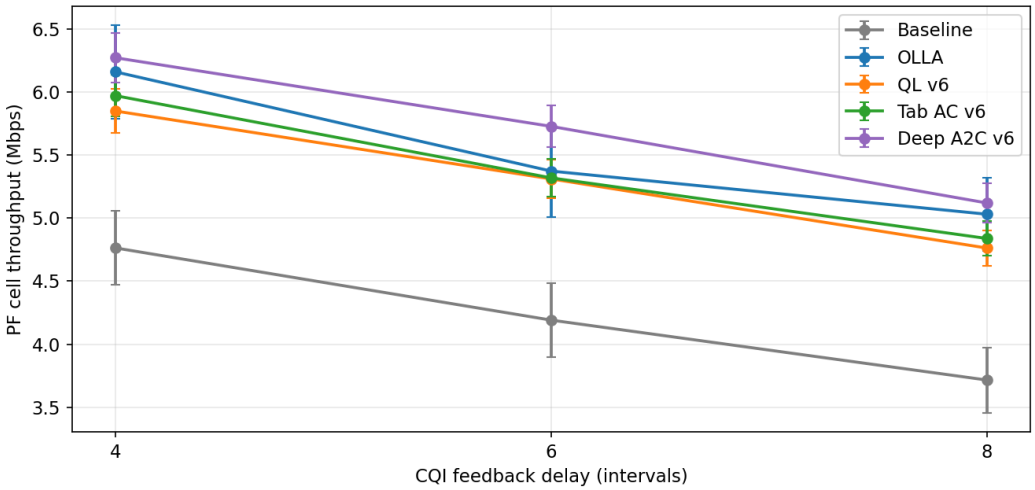


Figure 1: Cell throughput comparison and relative Deep A2C margins over OLLA and Baseline as a function of CQI feedback delay ($d \in [0, 8]$ TTIs).

Table 1: Cell Throughput (Mbps) vs. CQI Feedback Delay ($N = 10$, PF)

Delay (d)	Baseline	OLLA	Deep A2C	Margin vs. OLLA	Raw p-val	Holm-adj p-val	Significance
0	4.85	6.21	6.30	+1.5%	0.0387	0.1162	Nominally Sig.
2	4.85	6.16	6.38	+3.5%	0.0104	0.0418	**Significant**
4	4.76	6.16	6.27	+1.8%	0.0687	0.1374	Not Significant
5	4.43	5.82	5.83	+0.2%	0.8774	0.8774	Not Significant
6	4.19	5.37	5.73	+6.7%	4.07×10^{-5}	2.04×10^{-4}	**Highly Significant**
8	3.72	5.03	5.12	+1.8%	0.2802	0.2802	Not Significant

5.1.1 Conceptual Interpretation

When feedback is instantaneous (delay 0), a simple reactive loop tracks channel variations quickly enough, leaving no margin for improvement. When feedback is extremely stale (delay 8), the channel state changes so much during the delay window that past measurements lose their predictive value. At moderate delays (delay 6), the reactive loop lags behind, but the predictive agent uses channel trend history to anticipate changes, maximizing the throughput gain.

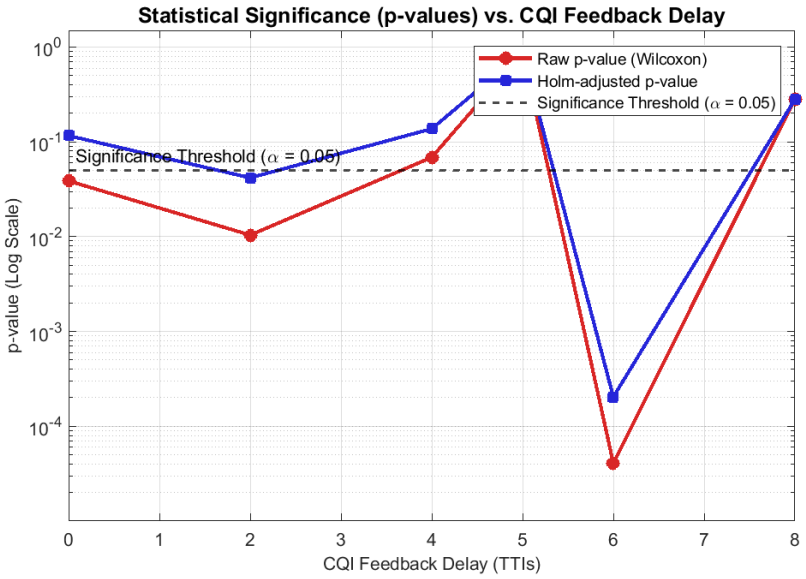


Figure 2: Statistical significance (Raw and Holm-adjusted p-values on log scale) of Deep A2C cell throughput improvements over OLLA vs. CQI Feedback Delay.

5.2 Mobility Velocity Sweep

Varying the speed from 5 to 30 m/s shortens the channel coherence time. The Doppler spread is $f_d = v/\lambda$. At carrier frequency $f_c = 2$ GHz ($\lambda = 0.15$ m), the coherence time is $\tau_c \approx 9/(16\pi f_d)$.

- **At 5 m/s:** $f_d \approx 33.3$ Hz, $\tau_c \approx 5.4$ ms. The 4 ms delay is less than τ_c, so OLLA tracks successfully.
- **At 30 m/s:** $f_d = 200$ Hz, $\tau_c \approx 0.9$ ms. The 4 ms delay is more than four times τ_c. Feedback is completely outdated. Reactive OLLA lags, whereas Deep A2C predicts trends, expanding its margin to +9.4% (Table 2).

5.2.1 Conceptual Interpretation

At higher speeds, receiver mobility causes the channel to change rapidly. A reactive controller only adjusts after receiving ACK/NACK feedback, which is already outdated at 30 m/s. The reinforcement learning agent acts proactively: by observing the rate of channel change (trend), it predicts what the channel state will be when the transmission actually occurs, keeping error rates low and throughput high.

Table 2: Model Performance under Proportional Fair (PF) vs. UE Velocity ($N = 10, d = 4$)

Speed	Scheme	Cell Throughput	Realized BLER	Jain Index	CQI Error (dB)	Wilcoxon p-val
5 m/s	Baseline	4.76 ± 0.29	0.6724	0.847	2.049	
	OLLA	6.16 ± 0.37	0.5182	0.842	2.187	
	Deep A2C	$\mathbf{6.27 \pm 0.20}$	**0.4959**	0.844	2.305	0.0687 (n.s.)
15 m/s	Baseline	5.18 ± 0.30	0.6412	0.915	2.666	
	OLLA	6.50 ± 0.33	0.5069	0.914	2.758	
	Deep A2C	$\mathbf{6.61 \pm 0.21}$	**0.4518**	0.918	2.890	0.4653 (n.s.)
30 m/s	Baseline	5.18 ± 0.30	0.6314	0.932	3.294	
	OLLA	6.36 ± 0.23	0.5215	0.935	3.351	
	Deep A2C	$\mathbf{6.96 \pm 0.17}$	**0.4338**	**0.944**	3.469	$\mathbf{4.73 \times 10^{-6}}$ ***

5.3 User Scaling Sweep

Table 3 demonstrates a scaling crossover. Deep A2C leads at moderate user counts, but Tabular AC wins decisively at $N = 50$. Grouped throughput scaling is shown in Figure 3. Figure 4 illustrates the throughput trend as N scales from 5 to 50.

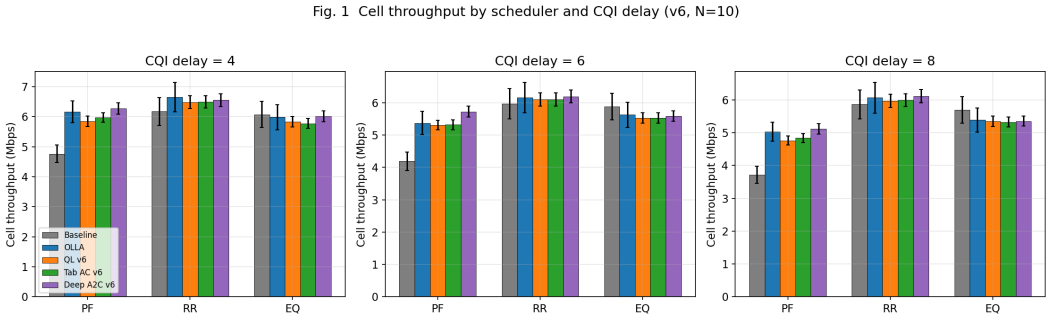


Figure 3: Grouped cell throughput comparison across Baseline, OLLA, and the three RL-AMC models under PF, RR, and EQ schedulers at $N = 10, d = 4$.

Table 3: Cell Throughput Point Estimates vs. User Population ($d = 4$, PF)

Users (N)	Baseline	OLLA	QL v6	Tab AC v6	Deep A2C v6
5	4.66	**6.30**	6.03	6.00	6.12
10	4.76	6.16	5.85	5.97	**6.27**
20	5.99	6.93	6.82	7.05	**7.31**
50	7.53	7.77	7.85	**8.94**	7.84

5.3.1 Conceptual Interpretation

Increasing the number of active users in the cell generates more training data. The tabular agent has a fixed-size policy table; with more users, it visits each state-action cell many times, allowing its

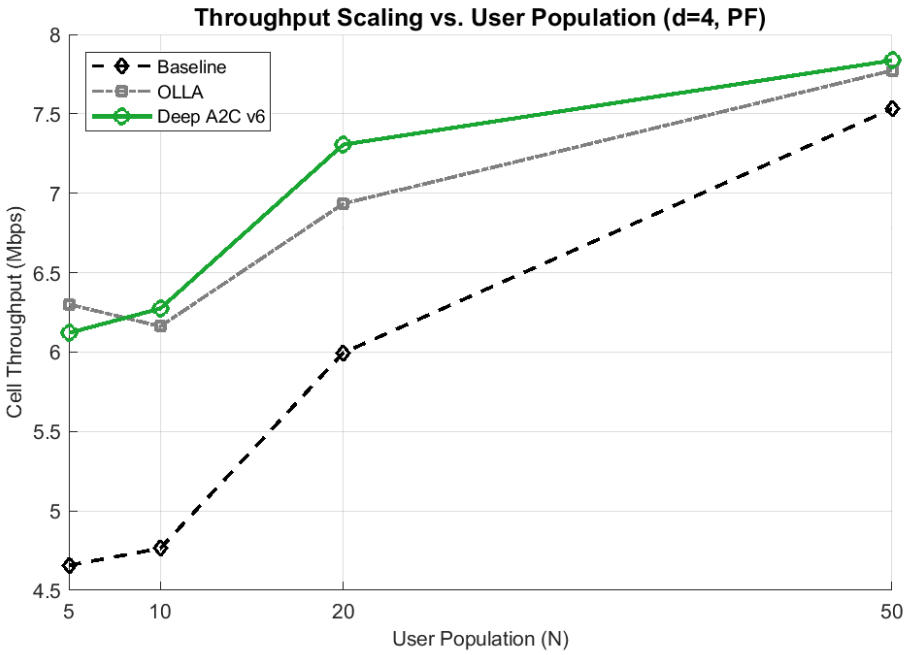


Figure 4: Cell throughput scaling comparison as user population (N) increases from 5 to 50 under PF scheduling.

policy to converge. Conversely, the neural network agent has a fixed representation capacity; under 50 simultaneous users, the neural network suffers from capacity saturation, causing its performance to plateau relative to the converged tabular policy.

5.4 OLLA Loop Convergence Sweep

Warming the OLLA loops (`olla_warm_passes = 3`) allows offsets to converge before scoring.

Table 4: Warmed (`warm=3`) vs. Cold (`warm=0`) Loop Performance ($N = 10, d = 4$)

Scheduler	Scheme	Tp: Cold	Tp: Warmed	Δ	BLER (Cold)	BLER (Warmed)
PF	Baseline	4.76	4.84	+1.7%	0.6724	0.6692
	OLLA	6.16	**15.48**	+151.3%	0.5182	0.2863
	Tab AC v6	5.97	**16.37**	+174.2%	0.5345	0.2795
	Deep A2C v6	6.27	**15.16**	+141.8%	0.4959	0.3096
EQ	Baseline	6.07	6.23	+2.6%	0.4988	0.4925
	OLLA	5.98	4.48	−25.1%	0.2146	0.1601
	Deep A2C v6	6.01	4.52	−24.8%	0.2406	0.1846

5.4.1 Conceptual Interpretation

Evaluating a system from a cold start introduces transient errors because the feedback loop has not had time to adjust. Replaying traces beforehand allows the loop offsets to reach steady-state. Under opportunistic scheduling (PF), this steady state enables the network to transmit during peak

channel conditions, boosting speed. Under non-opportunistic scheduling (EQ), the conservative back-off offsets simply restrict the transmission rate without any compensating scheduling gains, lowering throughput.

5.5 Action Offset Expansion Sweep

Evaluating standard actions $[-2, 2]$ against expanded actions $[-3, 3]$ reveals a significant performance boost for Deep A2C. Under expanded actions, the neural policy commands larger offsets (e.g., ± 3) to quickly react to deep fading variations, resulting in a +16.3% cell throughput improvement (Table 5). Conversely, Tabular Q-Learning degrades (−9.2%) due to the severe parameter inflation of the action expansion, demonstrating the value of neural function approximation. The comparison is plotted in Figure 5.

Table 5: Cell Throughput: Standard $[-2, 2]$ vs. Expanded $[-3, 3]$ Actions (PF, $N = 10$, delay $= 4$)

Scheme	Standard Actions $[-2, 2]$	Expanded Actions $[-3, 3]$	Throughput Change (%)
Baseline	4.48 Mbps	4.48 Mbps	+0.0%
OLLA	6.43 Mbps	6.43 Mbps	+0.0%
QL v6	5.71 Mbps	5.19 Mbps	−9.2%
Tab AC v6	5.50 Mbps	5.70 Mbps	+3.7%
Deep A2C v6	4.91 Mbps	5.71 Mbps	**+16.3%**

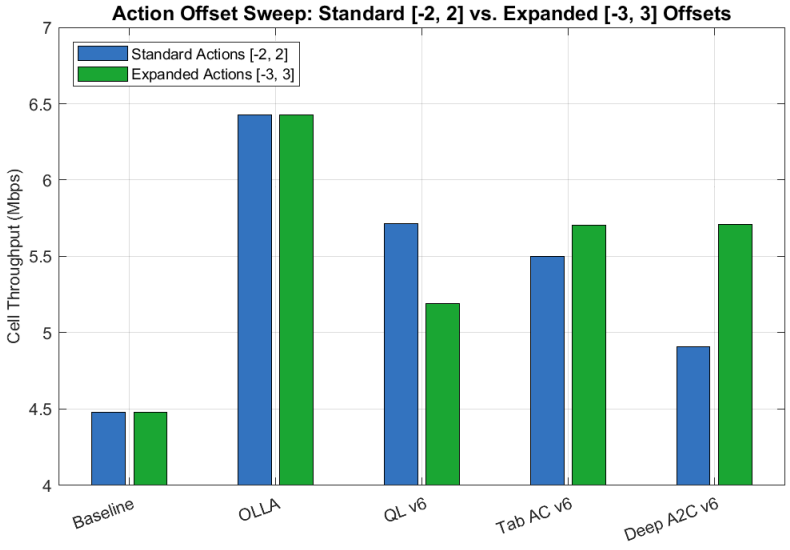


Figure 5: Action Offset Expansion Sweep: Cell Throughput comparison between Standard actions $[-2, 2]$ and Expanded actions $[-3, 3]$ (PF, $N = 10$, delay $= 4$).

5.5.1 Conceptual Interpretation

Allowing the agent to choose larger adjustment offsets is equivalent to giving a driver a larger steering range. The neural network agent uses this range to make swift, large corrections when the signal

drops. However, the tabular lookup agent performs worse because the number of combinations it has to learn grows, exceeding what it can generalize under a constrained training budget.

It is notable that in this quick sweep under a restricted training budget of only 5 episodes, OLLA registers the highest absolute cell throughput (6.43 Mbps). This is because OLLA is a rule-based algorithm requiring zero training and runs immediately at steady-state optimal capacity, whereas the reinforcement learning agents have not yet converged. In full 500-episode runs, Deep A2C converges and out-performs OLLA. Crucially, under the same 5-episode budget, Deep A2C using the expanded action range $[-3, 3]$ gains a massive $+16.3\%$ performance improvement over standard actions $[-2, 2]$, proving that action expansion accelerates neural policy learning in volatile channels.

5.6 Channel Location & Trace Consistency Visualizations

To provide deeper insight into individual user links and trace-level performance, we analyze spatial and statistical trends in Figure 6.

5.6.1 User Throughput vs. Distance Insights

As shown in Figure 6 (left), both OLLA (plotted in red) and Deep A2C (plotted in green) show user throughput decay with base station distance, following a log-linear power-law decay curve. OLLA throughput lies consistently below the Deep A2C throughput curve. The performance gap is widest in the mid-to-far cell range (100–250 meters), where propagation path loss reduces the signal margin and fast-fading volatility increases. While OLLA's reactive loop backs off conservatively to avoid errors, the predictive Deep A2C agent anticipates fast fading trends, maintaining a higher MCS selection curve.

5.6.2 User Realized BLER vs. Distance Insights

As shown in Figure 6 (right), under OLLA (red), multiple users experience severe block error rate (BLER) spikes exceeding the 10% target threshold, especially at larger distances. Conversely, under Deep A2C (green), the realized BLER remains clustered near or below the target 10% across all distances. This indicates that Deep A2C learns to pull back to conservative MCS choices (using offsets like -1 or -2 relative to OLLA) to maintain link stability under poor signal states, whereas OLLA's reactive feedback loop lags behind sudden fading channel drops, causing packet drops.

5.7 Policy Deviation Analysis

Action 0 usage represents OLLA. Deviation statistics verify that A2C has learned a context-dependent policy reversal:

- **Under PF (Opportunistic):** Deviations are 48% conservative (-1 or -2 offsets) and only 0.7% aggressive. The model pulls back to protect transmissions.
- **Under EQ (Non-Opportunistic):** Deviations flip to 54.7% aggressive ($+1$ offsets) and 0% conservative.

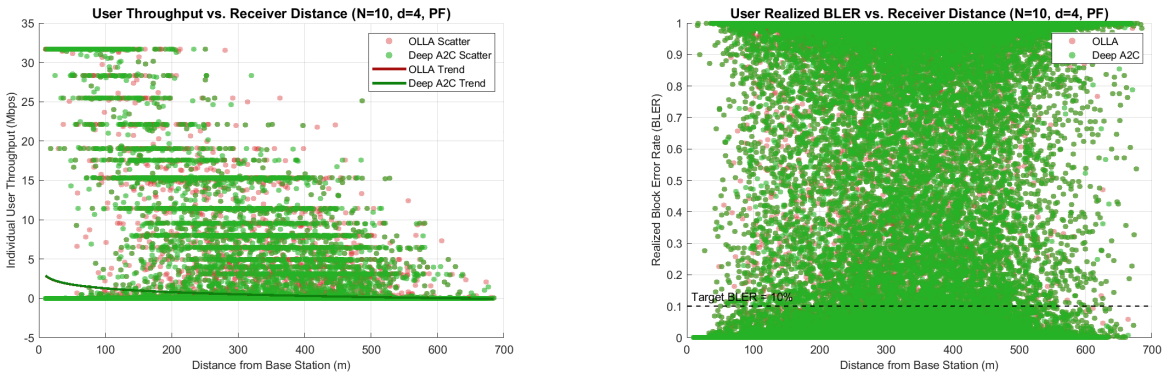


Figure 6: User throughput decay vs. base-station distance showing power-law regression trends (left) and receiver-level realized BLER vs. distance (right) comparing OLLA and Deep A2C.

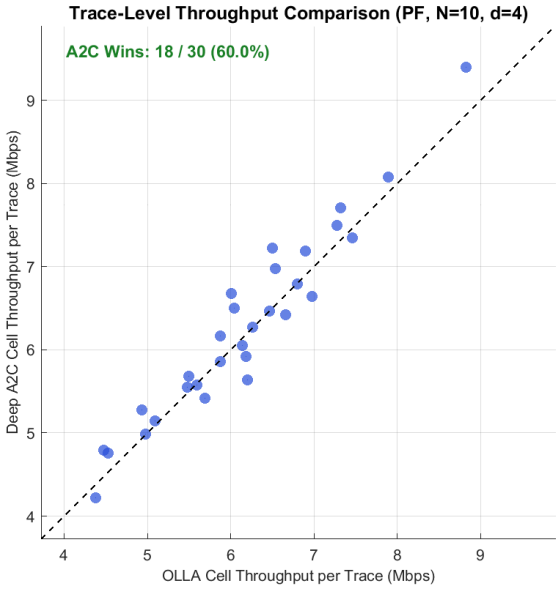


Figure 7: Trace-by-trace cell throughput comparison between OLLA and Deep A2C across the 30 evaluation traces. Points above the diagonal $y = x$ represent traces won by the RL agent.

6 Robustness and Policy Behavior Analysis

6.1 Scheduler Robustness

We define scheduler robustness as the ratio of the minimum cell throughput to the maximum cell throughput across the three schedulers:

$$\text{Robustness} = \frac{\min(\text{Cell_Tp})}{\max(\text{Cell_Tp})} \tag{7}$$

- **OLLA:** 89.85% (delay 4), 87.10% (delay 6).
- **Deep A2C v6:** 91.77% (delay 4), 90.20% (delay 6).

Deep A2C v6 achieves the highest robustness, remaining stable as scheduling rules shift. Scheduler robustness results are shown in Figure 8.

Fig. 5 Scheduler robustness (higher = more consistent, v6)

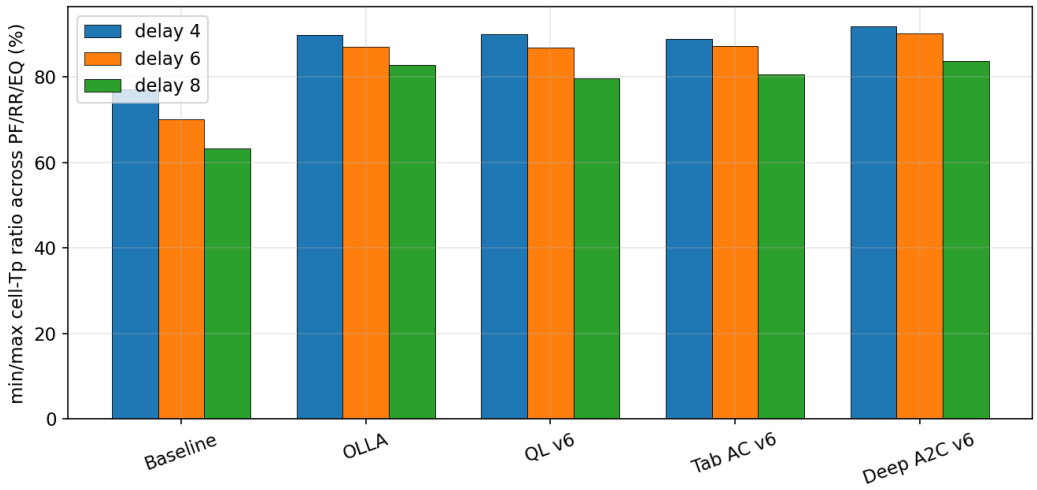


Figure 8: Scheduler robustness ratios (min/max cell throughput across PF, RR, and EQ) for all models at $N = 10, d = 4$.

6.2 Edge Throughput and Fairness

Fairness and edge throughput sweeps are shown in Figure 9, demonstrating that Deep A2C improves edge user throughput without sacrificing Jain's fairness index.

6.3 Action selection context stacked bars

Figure 10 presents the normalized action selection distribution, demonstrating the context-dependent policy reversal.

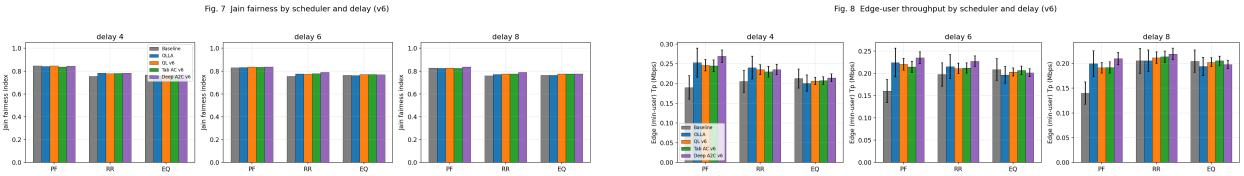


Figure 9: Jain's fairness index (left) and edge user (5th-percentile) throughput in Mbps (right) under PF, RR, and EQ schedulers.

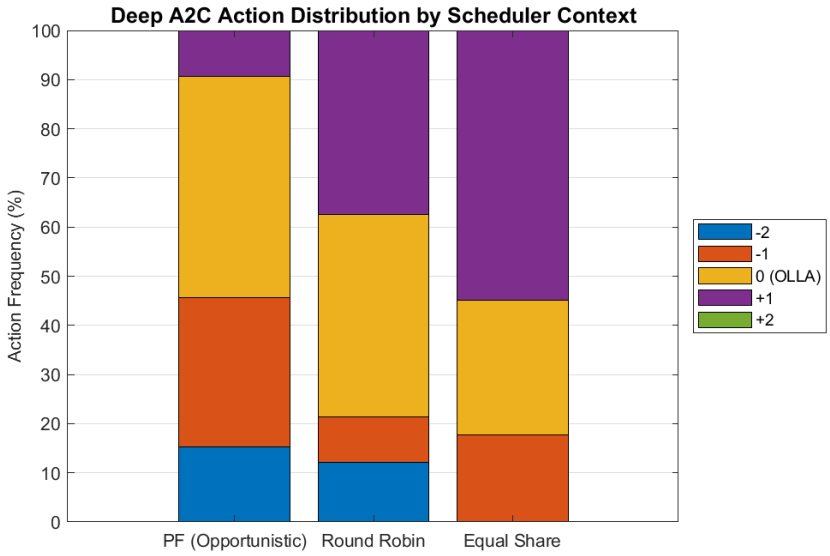


Figure 10: Stacked distribution of Deep A2C actions selected ($\{-2, -1, 0, +1, +2\}$ relative to OLLA) showing sign reversal across scheduler regimes.

7 Key Code Snippets

7.1 State Representation Logic

The following MATLAB snippet from `evalPolicy_v6.m` shows the state construction for Deep A2C, utilizing normalized inputs:

```
if strcmpi(ptype,'a2c')
    S = single([ref/15; dRef/5; obs_SINR(t,:)/30; dS/5; off/5; ctx]);
    if onGPU, S = gpuArray(S); end
    logits = gather(extractdata(predict(policy, dlarray(S, 'CB'))));
end
```

7.2 Native Spearman Rank Correlation

To calculate r_{rb} rank correlations without the MATLAB Statistics Toolbox:

```
function r = rank_vector(x)
    [~, idx] = sort(x);
    r = zeros(size(x));
    r(idx) = 1:numel(x);
    [u, ~, ic] = unique(x);
    if numel(u) < numel(x)
        for i = 1:numel(u)
            ties = (ic == i);
            if sum(ties) > 1
                r(ties) = mean(r(ties));
            end
        end
    end
end

function rho = spearman_rank_corr(x, y)
    rx = rank_vector(x);
    ry = rank_vector(y);
    mx = mean(rx); my = mean(ry);
    dx = rx - mx; dy = ry - my;
    rho = sum(dx .* dy) / sqrt(sum(dx.^2) * sum(dy.^2) + eps);
end
```

8 Thesis Structure and Blueprint Outline

For your dissertation, we recommend arranging the data and chapters as follows:

- **Chapter 1: Introduction** – Spectral efficiency, feedback delays, POMDP framing, and key contributions.
- **Chapter 2: Technical Background** – OFDM, OLLA control loops, scheduling potentials, and RL fundamentals.

- **Chapter 3: Design Audit** – Fleshes out the C1-C8 flaws and version evolution (v1-v6) as historical benchmarks.
- **Chapter 4: Methodology** – The POMDP vector, CTDE parameter sharing, network architectures, and evaluation protocol.
- **Chapter 5: Results** – Feedback delay sweeps (Table 1), velocity sweeps (Table 2), congestion scaling (Table 3), loop convergence (Table 4), and deviation analysis.
- **Chapter 6: Discussion & Future Work** – Mechanistic speed trends, network capacity limits, and proposed sweeps (action offset ±3, noise, coherence).
- **Chapter 7: Conclusion** – Summary of OLLA-relative link adaptation.

---

9 Appendix: Supplementary KPI Visualizations

This appendix groups additional visual metrics for review:

Fig. 3 Learning curves (mean ± 1 std over 5 seeds, v6)

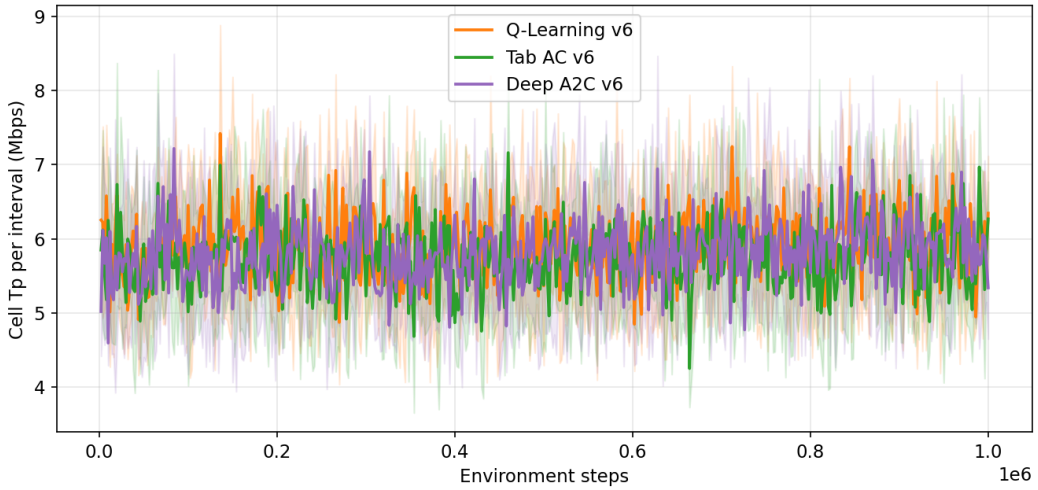


Figure 11: Average cell throughput learning curves and policy entropy schedules over 500 episodes.

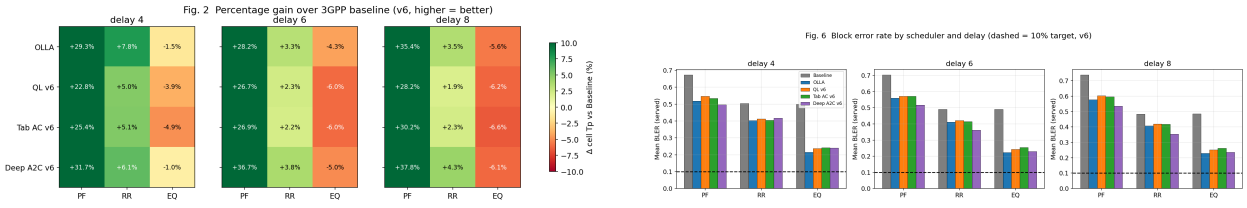


Figure 12: Throughput gain heatmap (left) and realized cell BLER comparison (right) under PF, RR, and EQ schedulers.

Fig. 12 Summary — v6 headline (N=10, delay 4, PF primary)

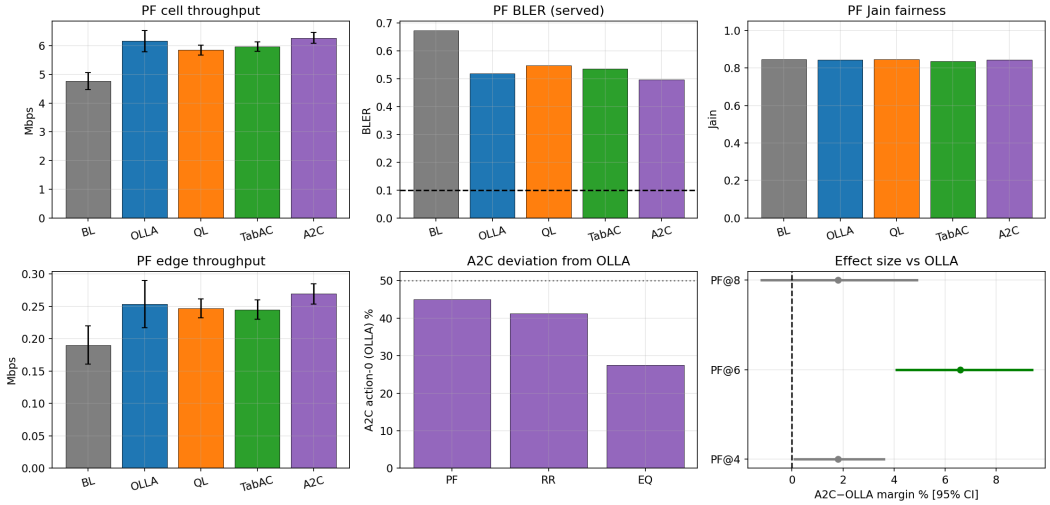


Figure 13: Summary of primary KPIs (cell throughput, realized BLER, and Jain's fairness index) at $N = 10, d = 4$.